What makes a *harness* a *harness*: necessary and sufficient conditions for an agent harness

Sanderson Oliveira de Macedo
Federal Institute of Goiás
sanderson.macedo@ifg.edu.br

June 8, 2026

Abstract

The term *agent harness* now circulates widely in software engineering with generative artificial intelligence. It names the layer that wraps a language model and turns it into a coding agent able to act on a repository. The usage is loose and polysemous. Sometimes the term denotes the whole product (Claude Code, Codex CLI); sometimes it denotes the evaluation scaffold that runs an agent against tasks (the SWE-bench harness); sometimes it gets conflated with an agent framework, an SDK, an IDE plugin, or an orchestrator. What is missing is a reference definition that works as an instrument, one that includes and excludes cases consistently. We build that definition through a conceptual analysis that combines works with persistent identifiers and primary grey-literature sources, such as official documentation, glossaries, and engineering reports. We reconstruct the genealogy of the term, from the horse's tack to the classic *test harness*, to the machine-learning *evaluation harness*, and finally to the *agent harness*. We then propose a constitutive definition that states the necessary and sufficient conditions for a system to be an agent harness, we operationalize it as an inclusion and exclusion test, and we draw the boundary of the concept against an agent framework, an agent SDK, an IDE plugin, an eval harness, and an orchestrator. We apply the definition to six real harnesses (Claude Code, Codex CLI, Aider, Cline, OpenHands, and SWE-agent) and to deliberate edge cases; the test includes and excludes consistently. We close with a research agenda organized by design tension axes. The contribution is an operational definition of agent harness, with a shared vocabulary, able to guide engineering practice and the scientific comparison of agentic systems.

Keywords: agent harness; coding agent; software engineering with generative AI; agent-computer interface; context engineering; operational definition.

1 Introduction

Picture a draft horse. It has power to spare. What it lacks is direction: loose, it bolts; tied to nothing, it moves no load at all. What turns brute force into useful work is the tack, the set of straps and belts that ties the animal to the cart, transmits the pull, steers the course, and stops the bolt. The tack does not replace the horse, and it does not compete with it. It works on a different layer, the engineering that channels power safely. English fixed this image in the verb *to harness*, and the image fits an object that has become central in recent software engineering: the layer that wraps a language model and turns it into an agent able to write code.

Large language models are no longer mere text generators. They have become the engine of systems that decide, call tools, and act on the environment [4, 50, 64]. The decisive turn was to couple reasoning and action. Instruction alignment made models follow requests [31]; chain-of-thought, tree-of-thought, and self-consistency improved deliberation [51, 58, 47]; patterns such as ReAct came to interleave thinking and acting in a loop; and external tools, from function

calling to browsing, extended the model's reach beyond its context window [59, 39, 36]. On that base, coding agents were born. They take a task in natural language, read a repository, edit files, run commands, and verify their own work [55, 46, 61]. The literature already treats these agents as a research line of their own, with surveys that systematize their architectures, capabilities, and limitations [44, 53, 24].

In this emerging vocabulary, one word keeps surfacing to name the infrastructure that surrounds the model: *harness*. The official Claude Code documentation defines the *agentic harness* as the set of tools, context management, and execution environment that turns a model into a capable coding agent, and sums up the relationship in one sentence: Claude Code is the harness, and Claude is the model inside it.[1] The Hugging Face agent glossary records the same broad usage, naming products such as Claude Code, Codex, and Antigravity as harnesses, that is, everything that is not the model.[2] The term travels without a passport. In classic machine learning, *harness* denotes an evaluation suite; SWE-bench itself, today the reference evaluation for coding agents, calls its execution scaffold a harness [19]. In software engineering, *test harness* predates generative AI. And in agentic usage, *harness* sometimes means the whole product, sometimes the orchestration layer, sometimes gets conflated with an agent framework, an SDK, an IDE plugin, or an orchestrator. The polysemy is not a terminological detail. It blocks us from comparing systems, attributing design merit, and accumulating knowledge about what, after all, makes an agent trustworthy.

The practical consequence of this confusion has a name. An agent hits an obstacle and, instead of admitting it, reports a success that did not happen. The common reflex is to tweak the prompt. But language models, when cornered, tend to produce the answer that seems to satisfy the request, true or not [5], and politely instructing the model not to do so is the weakest of controls. The robust solution is engineering around the model: detect the divergence between what the agent claims and the real state, verify that state deterministically, and run the sensitive parts with ordinary code rather than trust the model's word. It is that layer, not the prompt, that separates a demo that impresses from a product that holds up. Giving it a precise name is the object of this article.

The concept is central in practice, yet the academic literature rarely defines it head-on. Surveys of software agents describe components (memory, tools, reasoning loop) without consolidating what constitutes the harness as a unit [17, 24, 29]; work on the agent-computer interface treats one facet, tools, and stops there [55]; context engineering, now a topic of its own, covers another facet, context management, and does not close the definition [30]. The reference definition that works as an instrument is missing: one that says, given a concrete system, whether or not it is an agent harness, and why.

This article delivers that definition. The central question is direct: what is the reference definition of *agent harness*, and how do we make it an instrument that discriminates cases rather than a slogan? To answer it, we organize five contributions around five research questions. **RQ1 (genealogy):** where does the term *harness* come from and how did its sense migrate to agentic usage? **RQ2 (constitutive definition):** what conditions are necessary and sufficient for a system to be an agent harness? **RQ3 (boundary):** how does the definition distinguish a harness from an agent framework, an SDK, an IDE plugin, an eval harness, and an orchestrator? **RQ4 (application):** how do real harnesses instantiate the constitutive conditions? **RQ5 (agenda):** which design axes remain open?

The contributions are, in order: a genealogy that reconstructs the term's sense migration (Section 3); a constitutive definition operationalized as an inclusion and exclusion test (Section 4);

[1] Official Claude Code documentation, entry *Agentic harness*: https://code.claude.com/docs/en/glossary.

[2] Hugging Face, *Harness, Scaffold, and the AI Agent Terms Worth Getting Right*: https://huggingface.co/blog/agent-glossary.

a boundary delimitation against five neighboring concepts (Section 5); the application of the definition to six real harnesses, showing that it classifies consistently (Section 6); and a research agenda organized by tension axes (Section 7). Section 2 positions the article against the literature.

Positioning. This article is deliberately definitional. It does not propose a new agent, does not measure benchmark performance, and does not recapitulate taxonomies of AI systems for software engineering that the literature already has. Its product is conceptual: a shared vocabulary and a reproducible test of membership in the concept. The wager is simple. Terminological clarity is a precondition for cumulative science in a field that, today, calls too many things a *harness*. A note on method: this is a work of conceptual clarification, not a survey or a usage count; each source with a verified DOI enters as a formal citation, while product documentation and glossaries, where the term is actually used, enter as footnotes with a URL.

2 Related Work

The literature around the agent harness is vast, but it touches the concept sideways, by facets, and never closes it. Surveys of language agents describe architectures in terms of perception, memory, planning, and action, and recent studies shift the focus to evaluating agentic behavior [44, 53, 43, 60, 6]; they treat the agent as the object and push the infrastructure into the background. In software engineering, maps of model use across the lifecycle [17, 63, 24], agentic runtime systems [55, 46, 61, 54, 7, 1], multi-agent architectures [35, 16, 52, 21, 9, 13, 14, 66], and studies of code action and collaboration [45, 12, 33] each describe their own system, without the cross-cutting axis. Another front goes deep into isolated components, the agent-computer interface, context engineering, memory, tool use, and self-correction [30, 25, 20, 62, 32, 48, 37, 22, 42, 40, 28, 41]. And the control front deals with guardrails, attacks, and benchmark evaluation [38, 18, 3, 27, 10, 49, 67, 19, 56, 26, 65, 11, 57, 23, 2]. Each piece is part of the harness; none, alone, is the harness. What is missing is a reference definition that unites these fronts into a concept with sharp boundaries and a membership test. That is the gap we occupy. This article does not compete with the system catalogs, which say what; it answers the *what it is*, prior to any catalog. The systems reappear as empirical material in Section 6, and the evaluation sense of the term is separated from the agentic sense in Section 5.

3 Genealogy of the Term (RQ1)

To define a term well, start with where it came from. The word *harness* carries a largely stable metaphor across centuries and domains. Reconstruct that metaphor and you see why it was repurposed to name the infrastructure of agents. We answer RQ1 by tracing four stations: the etymological origin, the *test harness* of software engineering, the *evaluation harness* of machine learning, and the *agent harness*.

From armor to tack

In English, *harness* comes from the Old French *harneis* (also *harnois*, twelfth century): equipment, armor, the set of war gear. Its origin is uncertain, possibly the Old Norse *hernest*, the provisions of an army. Around 1300, the word enters English meaning fighting equipment and armor, including that of a knight *in full harness*, in complete armor. By the early fourteenth century the non-military sense leaves the battlefield for the stable: *harness* comes to name the set of straps and belts that connects a draught animal to a cart, the tack.[3] The verb *to harness*, attested

[3]Dates and senses follow historical dictionaries of English: *Online Etymology Dictionary*, entry *harness* (https://www.etymonline.com/word/harness); *Oxford English Dictionary*, entries *harness, n.* and *harness, v.*; and *Merriam-Webster*, entry *harness* (https://www.merriam-webster.com/dictionary/harness). The fighting-equipment sense is dated to around 1300 and the draught-animal sense to the early fourteenth century.

around 1300 in the sense of putting tack on an animal, only takes on the figurative sense that survives today, channeling a force to make it useful, in the late seventeenth century, as in *harness the wind* or *harness solar energy*.[4] The idea never changes: take a brute force and channel it safely to produce work. That is the parent metaphor of everything that follows. Force without direction on one side; infrastructure that connects, controls, and measures on the other.

Figure 1 translates the metaphor to artificial intelligence. The horse is the model: power able to generate text and decide, but blind to the world and prone to bolting [5]. The cart is the task to be delivered. The tack is the engineering layer that connects the model to the task, controls the execution, measures what happens, and monitors the result. The agent is the harnessed horse; the harness is the tack.

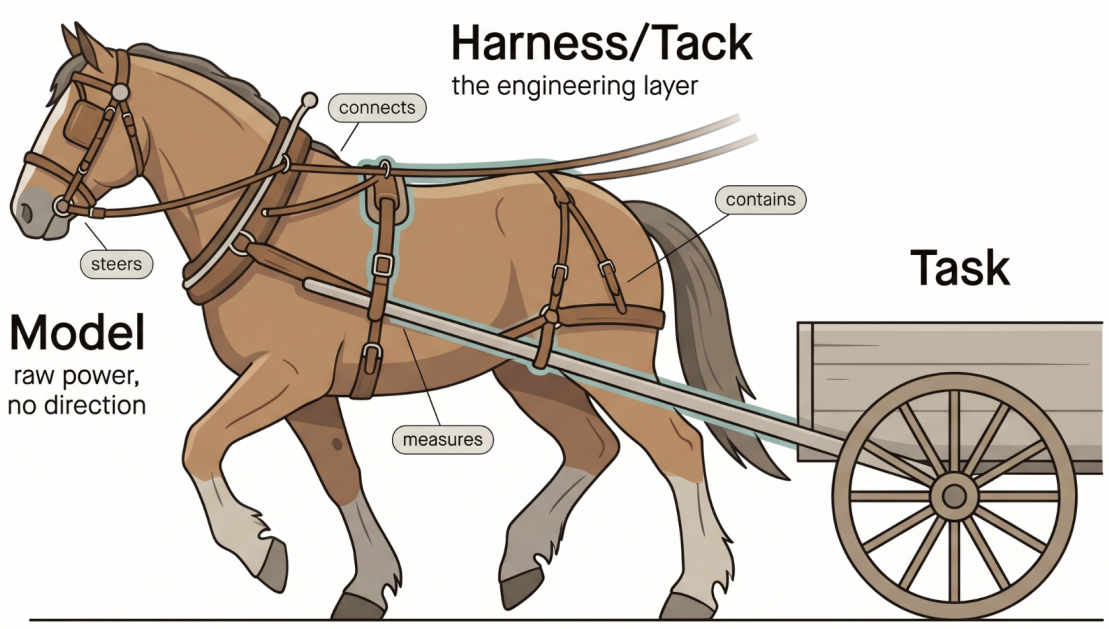

Figure 1: The parent metaphor of the harness. The language model is the horse, brute force with no direction of its own; the task is the cart to be pulled; the harness is the tack, the engineering layer that connects, steers, measures, and contains, turning uncontrolled power into trustworthy work.

The classic test harness

The term was not born with AI. In software testing, a *test harness* is the set of scripts, mocks, stubs, and infrastructure that runs tests in a controlled and observable way. The tack metaphor was already there. Just as the tack connects and controls the horse, the test harness ties the code to an infrastructure that observes and controls its behavior during execution. This usage is well established in engineering, predates language models, and is the first reason the word sounds familiar to anyone who programs.[5]

The machine-learning evaluation harness

From the test harness, close relatives derived in the world of models: the *evaluation harness* and the *benchmark harness*, for measuring performance. Here the harness is an evaluation suite. One

[4] The figurative sense of *to harness*, controlling a force to extract useful work from it, is dated to the 1690s by the *Online Etymology Dictionary* (entry *harness*, verb sense) and recorded in the same sources above.

[5] The classic sense of *test harness* is described in consolidated software-testing vocabularies, such as the *ISTQB Glossary* (`https://glossary.istqb.org`).

connects the model to the harness, runs a set of standardized tasks, and measures how well it did. The focus is to evaluate, to assign a grade. That sense now dominates agent evaluation: SWE-bench calls its task executor a harness [19], and benchmarks such as AgentBench, WebArena, Mind2Web, and τ-bench follow the same logic, a scaffold that runs the system against scenarios and collects the result [26, 65, 11, 57]. It is a harness that observes from the outside, after the work has happened.

The agent harness

The *agent harness* inherits the name and the metaphor, and widens the scope. It does not just evaluate at the end. It controls, limits, verifies, and corrects the execution while it happens. The shift tracks a change of era. In classic machine learning the system was passive: input went in, output came out, and the harness measured quality. In the agentic regime the system acts, calls tools, browses, writes files, authenticates [55, 46]. A system that acts needs more than evaluation at the end. It needs control along the way. Table 1 summarizes the two senses that coexist today, whose confusion motivates this article.

Context	**What *harness* means**
Software testing (classic)	Scripts, mocks, and stubs that run code in a controlled and observable way
Model/agent evaluation	A suite that runs the system against standardized tasks and measures the result
Agent engineering (runtime)	The layer that controls, limits, verifies, and corrects the agent's execution while it happens

Table 1: The senses of *harness* in circulation. The first two observe from the outside; the third, the target of this article, acts during execution.

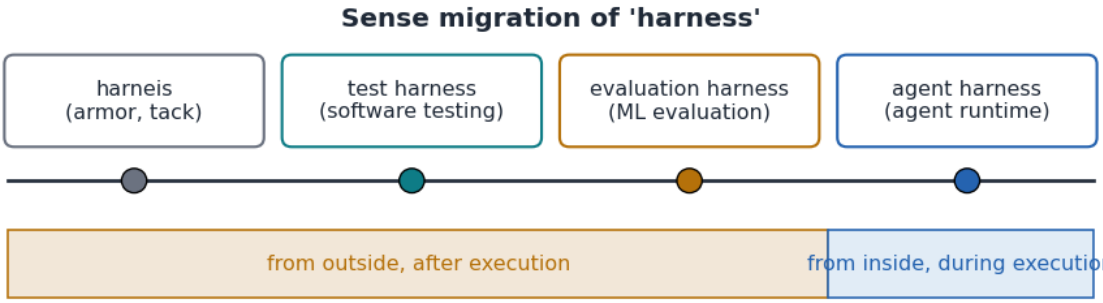

Figure 2: Timeline of the sense migration of *harness*. From the Old French *harneis* (armor and tack) to the software-engineering *test harness*, to the machine-learning *evaluation harness*, and to the agentic *agent harness*. The lower band marks when control acts: from outside and afterward, in the first three; from inside and during, in the last.

Figure 2 synthesizes this migration, from tack to agent, as a timeline. The thread is single: in every station, a harness is the infrastructure that channels a force (the horse, the code under test, the model) to produce useful work under control. What changes is what is channeled and when control acts. In the agent harness it acts at runtime, and that change is what demands a definition of its own, which we build next.

4 Constitutive Definition (RQ2)

The genealogy tells us where the term comes from. What remains is to say what it designates today, and precisely. We define the agent harness by conditions, not by examples, and we turn that definition into a test that includes and excludes concrete cases: this is our answer to RQ2. A good constitutive definition enumerates what is necessary, drops what is incidental, and separates the concept from its neighbors [15].

We propose the following reference definition of the agent harness.

> *An agent harness is the runtime engineering layer that wraps one or more language models and turns them into an agent able to accomplish tasks over an external environment, by coupling to the model: (i) an agent loop that interleaves reasoning, action, and observation; (ii) a tool interface that lets the model perceive and alter the environment; (iii) context management that decides what enters and leaves the model's window; and (iv) control mechanisms, that is, limits, verification, and deterministic actions, that make the execution more trustworthy, auditable, and contained.*

A system is an agent harness if and only if it instantiates the four elements above at runtime. The definition states conditions, not cases. The temporal clause carries a lot of weight: the harness acts *during* the task, and that is what separates it from the evaluation harness, which acts afterward. We did not invent the four elements; they come from the mechanisms the technical literature treats as recurrent in agents that act. The reasoning and action loop [59, 41]. Tool use and the interface with the environment [39, 55, 34]. Context management and retrieval [30, 25, 20]. And control by verification and containment [28, 38].

Anatomy

Figure 3 opens the concept into components. At the center is the model, the engine. Around it, the harness assembles what the engine lacks to become a drivable vehicle: a *tool registry* (the catalog of what the agent can do, read a file, run a command, browse), a *context manager* (which compacts history and selects the relevant), a *memory* (which survives across steps and sessions [32, 62]), the *agent loop* (reason, act, observe), a *verifier* (which checks whether the task was actually accomplished, rather than accepting the model's word [28]), *retry logic* with eventual model switching (which re-runs transient failures), *observability* (an auditable record of what happened), *guardrails* (safety limits [18, 38]), and *deterministic handlers* (ordinary code for the parts too sensitive for the model). Not every harness ships all of those optional components. The four elements of the definition, though, are the core. Without them there is no harness.

Each condition is necessary, and that becomes clear when we take one element away at a time and watch what is left. Without an agent loop, the system answers once and stops; it is a generator, not an agent, and no agent survey would classify it as such [44]. Without a tool interface, the model neither perceives nor alters the environment; it stays trapped in its own window, unable to act on a repository [55]. Without context management there is no viable loop: every multi-step task accumulates observations, tool outputs, and test results that compete for the model's window, and deciding what feeds back into the model is constitutive of the loop, not a luxury reserved for long tasks; the literature confirms that the alternative, letting useful information dilute in the buildup of history, degrades the model [25, 30]. And without control mechanisms, the system acts, but no one can tell whether it did what it claims, nor is there a way to contain it; it is exactly the failure that motivates the topic, when an agent reports a nonexistent success and nothing verifies it [5, 28]. The four, together, are sufficient. Any system that satisfies T1 through T4 already exhibits the behavior that motivated the concept: channeling the model's brute force with control exercised at runtime. Once the four hold, nothing essential is missing, and there is no fifth condition to add. Memory, verification, observability,

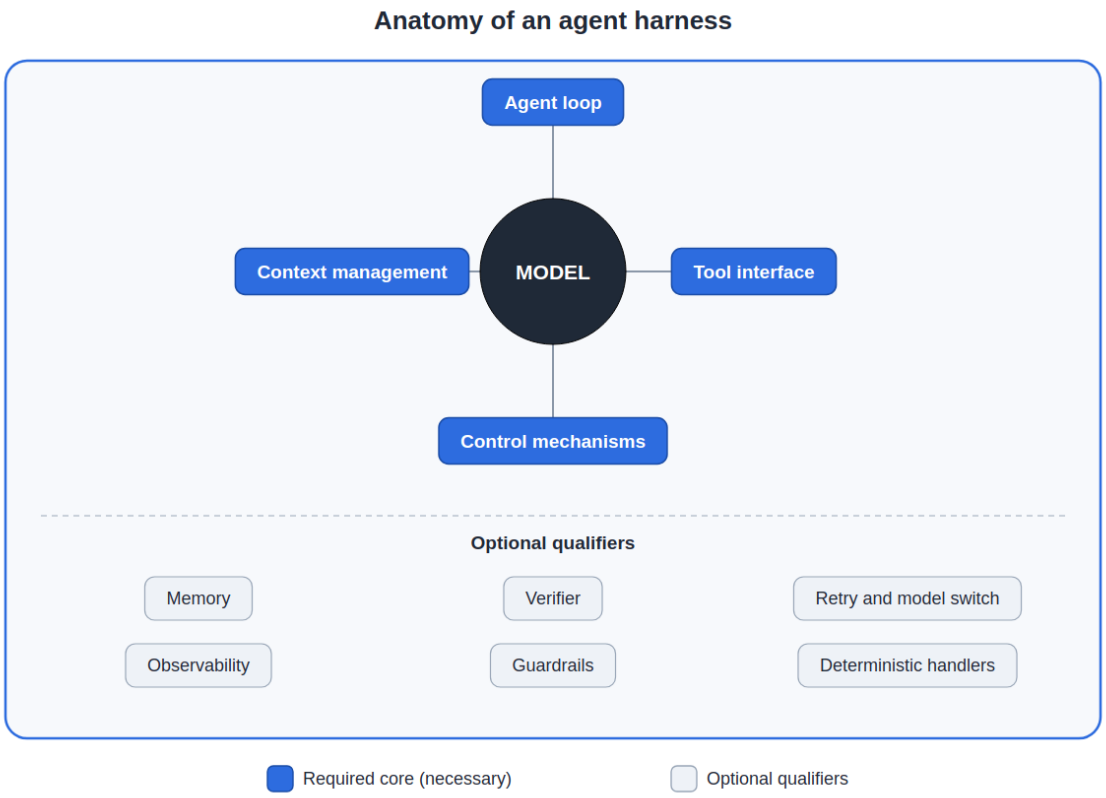

Figure 3: The anatomy of an agent harness. At the center, the model, the engine. Around it, the components that turn it into a trustworthy agent: tool registry, context manager, memory, agent loop, verifier, retry and model switch, observability, guardrails, and deterministic handlers. The four core elements (loop, tools, context, control) are necessary; the rest qualify the harness.

and the like are not new elements; they are specializations of T1 through T4, that is, more robust ways to keep the loop, perceive the environment, manage context, and control execution, exactly as the next paragraph shows when it separates what is core from what is incidental. None of the four, in isolation, dispenses with the others.

What is NOT necessary also has to be said, and the definition stays lean because it leaves the incidental out. An agent harness does not require multi-agent: a single model in a loop with tools and control is already a harness, and multi-agent architectures are a design choice, not a requirement [35, 52]. It does not require learning or fine-tuning: the harness is engineering around the model, not modification of it. It does not require a specific model, and that is a property, not a defect; a good harness extracts trustworthy work from diverse models, and model switching is a control mechanism, not a dependency. Nor does it require a user interface: a harness can be a faceless library. Pulling these items into the definition would make it too narrow and would throw out systems that are, legitimately, harnesses.

The inclusion and exclusion test

The definition becomes an instrument once we turn it into a decision procedure. Given a candidate system, ask, in order: (T1) does it maintain a loop that interleaves reasoning, action, and observation at runtime? (T2) does it offer the model an interface to perceive and alter an external environment? (T3) does it actively decide what enters and leaves the model's context? (T4) does it include at least one control mechanism, verification, limit, or deterministic action,

that does not depend on the mere obedience of the model? A system is an agent harness if it answers *yes* to T1 through T4. Fail any one and the system falls into a neighboring category; Section 5 shows which. Table 2 summarizes the test and anticipates what each failure implies.

Test	**Question**	**If the answer is *no***
T1	Is there a reasoning, action, and observation loop at runtime?	Single-pass generator or fixed pipeline; not an agent
T2	Is there a tool interface to perceive and alter the environment?	Isolated model or SDK that does not yet build the loop
T3	Is there active management of what enters and leaves the context?	Naive wrapper that dumps history; brittle on long tasks
T4	Is there at least one control mechanism independent of the model?	Demo without guarantee; trusts the model's word

Table 2: Inclusion and exclusion test derived from the constitutive definition. A system belongs to the concept if it answers *yes* to T1 through T4; each failure indicates the neighboring category into which the system falls.

Threshold and gradation are two caveats that keep the test from being read naively. The first is the threshold of each condition, especially T3. A wrapper that merely truncates history when it overflows the window does *some* context handling, but it does not satisfy T3 in the intended sense. The verifiable criterion is this: T3 is satisfied if the decision about what enters and leaves the context depends on the *content* of the task or of the current observation, and not merely on the *size* of the buffer. A mechanical cut by size fails the test; active, task-aware selection that decides what matters for the current step passes. T2 also has a verifiable criterion: T2 is satisfied if the interface lets the model *alter* an external environment (edit files, run commands), not merely read it or suggest text. Reading signals from the environment without being able to change it fails the test; an interface that acts on the environment passes. The same rigor holds for T4. The criterion is the following: a mechanism satisfies T4 if its effectiveness does *not* depend on the model choosing to cooperate. A single log *print* fails, because it does not alter the course of execution; a cap on tool calls or a deterministic check passes, because it contains or alters execution independently of whatever the model decides to do. The second caveat: membership is binary in its existence but gradual in its quality. A minimal loop that, at each step, re-runs the repository's test suite and declares success only if the suite passes satisfies T1 through T4 to the letter and is, in fact, an embryonic harness: re-running the tests is a genuine deterministic check that confirms the outcome regardless of whether the model claims to be done. What distinguishes it from Claude Code or OpenHands is not whether it belongs to the concept; it is the maturity of the mechanisms, above all the control ones. The test decides *whether* a system is a harness. The anatomy qualifiers measure *how* robust it is. Treating those two questions as one is the source of much of the terminological mess this article combats.

A harness and a guardrail must be contrasted, since the confusion between them is the most common of the topic. Guardrail and harness are not synonyms: the guardrail is a piece of the harness, not the whole. The difference is one of function. Guardrails *limit*: they restrict, block, validate, impose boundaries, such as a cap on tool calls, a cost limit, or the blocking of destructive commands [38, 18]. The harness, as a whole, *enables*: the context manager helps the agent remember, memory avoids repeating work, retry overcomes transient failures, the verifier confirms completion. The test question is single: is this limiting the agent, or helping it execute? If it limits, it is a guardrail. If it helps, it is a harness. Guardrails are inside the harness; the harness is not inside the guardrails. For that reason the guardrail is not one of the lateral neighbors of the boundary treated in Section 5, but a kind of control mechanism, part of T4: the relation here is part-whole, not that of two adjacent categories that get confused for each other. That asymmetry matters because it places control, not restriction, at the center of the concept.

5 Boundary Delimitation (RQ3)

A definition only discriminates if it separates the concept from its neighbors. Five concepts are routinely confused with the agent harness: agent framework, agent SDK, IDE plugin, eval harness, and orchestrator. We confront the harness with each one, always through the case that separates them, and we anchor that case in the inclusion and exclusion test of Section 4.

An agent framework differs from a harness by operating above it. An agent framework, such as AutoGen, CAMEL, AgentVerse, or the more widely adopted production frameworks CrewAI, LangGraph, Google's Agent Development Kit, and Agno,[6] offers abstractions to *compose* agents: roles, conversation protocols, collaboration patterns [52, 21, 9, 16]. The harness is the layer *below*: it makes an individual agent act reliably on the environment. A framework can use a harness beneath each agent it coordinates. A harness needs no framework at all, since a single agent already instantiates it. The confusion arises because many frameworks embed a minimal harness. Two cases separate them: a system that merely routes messages between personas, without a loop of action on an external environment, fails T2 and is a framework, not a harness, whereas a single agent that edits files and verifies the result, without any role composition, is a harness, not a framework.

An agent SDK separates itself from a harness by being raw material, not a finished product. An SDK delivers the building blocks, function calling, tool definitions, loop primitives, but does not impose a running agent. It is raw material [34, 37]. The harness is the product assembled from that material: it builds and runs the loop. A library that exposes the tool-calling primitive but leaves the developer to assemble the loop and the control fails T1, because there is no loop at runtime until someone writes it; it is an SDK. When that same library already delivers the closed loop, with context and verification, it has crossed the boundary and become a harness.

An IDE plugin is distinguished from a harness by closing no loop and not acting on the repository. An autocomplete plugin generates snippets from the cursor. It suggests code in the editor without maintaining task state, without acting on the repository, and without verifying the result [8]. It lacks the loop (T1) and the control (T4). The harness, even when it lives inside an editor, maintains the agent loop and acts on the environment. Completing the line where the cursor sits is a plugin; taking a task, planning, editing multiple files, running tests, and checking is a harness, even though both live in the same IDE.

The eval harness is the most direct name collision with the agent harness, and the easiest one to slip on. The eval harness, such as the one in SWE-bench, runs an agent against a set of tasks and *measures* the result after each task has ended [19, 26, 57]. One might object that it too has a loop, since it runs many tasks in sequence. The objection conflates two distinct loops. The eval harness's outer loop iterates over *tasks* and collects scores. The loop of T1 is the inner loop of reasoning, action, and observation that conducts a *single* task. The eval harness does not close that inner loop: it outsources it to the system under test, observes the outcome, and assigns a grade. From the point of view of a single task, it fails T1, because it does not decide the next step from the observation of the previous one, it only records what the system under test decided. The temporal clause seals the distinction: the eval harness acts *after* the fact, the agent harness *during* the fact. One judges the race, the other is the vehicle that runs it.

An orchestrator separates itself from a harness by following a fixed graph rather than an adaptive loop. An orchestrator, in the workflow sense, chains predefined steps in a fixed graph, a deterministic pipeline of steps [40]. The harness may contain orchestration, but what characterizes it is the adaptive loop: the next step depends on the observation of the previous one, not on

[6] Official sources for the production frameworks, cited here as grey literature: CrewAI (`https://crewai.com`); LangGraph (`https://www.langchain.com/langgraph`); Google's Agent Development Kit (`https://adk.dev`); Agno (`https://github.com/agno-agi/agno`).

a fixed graph. A pipeline that always runs A, then B, then C, without letting the observation alter the course, is an orchestrator; a loop in which the agent decides the next step from what it observed is a harness. Whatever only chains fixed steps fails T1, the adaptive loop.

As a boundary synthesis, Table 3 consolidates the five distinctions through the lens of the inclusion and exclusion test. The test is binary: each condition takes only *yes* or *no*, with no intermediate values, and a system passes a condition only if it satisfies it fully. Read vertically, it shows that no neighboring concept satisfies T1 through T4 at once in the sense of the agent harness. The boundary is not arbitrary: each neighbor fails, in binary fashion, at least one identifiable condition. That is what makes the definition an instrument, and not a label.

Concept	**T1**	**T2**	**T3**	**T4**	**What separates it from the harness**
Agent harness	yes	yes	yes	yes	(it is the reference concept)
Agent framework	no	no*	no	no	Composes agents; the loop, tools, context, and control belong to the harness it coordinates, not to the framework itself
Agent SDK	no	yes	no	no	Offers blocks; assembles no loop at runtime
IDE plugin	no	no	no	no	Suggests code from the cursor without altering the repository or closing a loop
Eval harness	no	yes	no	no	Runs commands and applies patches, but measures from outside, after execution: fails T1
Orchestrator	no	yes	no	no	Chains fixed steps; selection is not observation-driven and the loop is not adaptive (fails T1 and T3)

Table 3: Boundary of the agent harness against five neighboring concepts, through the lens of the T1 to T4 test. *A framework may embed a harness in each agent it coordinates; the *no* mark refers to the framework itself, not to the embedded harness.

6 Application to Real Harnesses (RQ4)

A definition that does not classify real cases is decorative. We apply the inclusion and exclusion test to six real harnesses and answer RQ4. The six were chosen to stress the definition across systems that differ from one another: Claude Code, Codex CLI, Aider, Cline, OpenHands, and SWE-agent. We classify each one from public documentation and, where available, from the academic description with a persistent identifier. We do not want to rank them. We want to show that the definition discriminates consistently across systems that differ widely from one another.

Claude Code is a terminal agent whose own documentation names the case a harness: it takes tasks, reads and edits files, runs commands, and maintains a reasoning and action loop, with context management and interception of dangerous operations at runtime.[7] It satisfies T1 (agent loop), T2 (file and shell tools), T3 (context compaction and selection), and T4 (runtime guardrails that ask for confirmation before destructive actions). Classification: agent harness, with all four core elements present.

Codex CLI is an open-source terminal agent that operates over the local repository with an action loop and a tool interface, offering permission modes that grade how much the agent can do without approval.[8] It satisfies T1 through T4, with control appearing explicitly in the permission modes (a form of runtime guardrail). Classification: agent harness.

Aider is a pair programmer in the terminal, integrated with version control. It edits files and commits changes with automatic commit messages, keeping a repository map as context

[7]Official Claude Code documentation: https://code.claude.com/docs. The *Agentic harness* entry explicitly states that the product is the harness and the model is what is inside it.

[8]Codex CLI documentation and repository: https://github.com/openai/codex.

management.[9] It satisfies T1 (an edit and correct loop), T2 (file editing and execution), T3 (a repository map that selects relevant context), and T4 (version-control integration, which makes each step auditable and reversible). Classification: agent harness, with auditability anchored in version control.

Cline is an agent that lives inside the editor (an IDE extension), with a plan and act loop, an interface to read and write files and run commands, and a flow that asks for human approval before sensitive actions.[10] The case is instructive because it lives in an IDE, like a plugin, but it is not an autocomplete plugin: it maintains the loop (T1) and acts on the environment (T2), with managed context (T3) and human approval as control (T4). Classification: agent harness, and not an IDE plugin, precisely because it passes T1 and T4, where the autocomplete plugin fails.

OpenHands is an open platform for development agents, described academically. Its agents write code, use the command line, and browse the web, all inside an isolated environment (a sandbox) [46]. It satisfies T1 through T4, and sandboxed execution reinforces control as a strong form of deterministic containment. Classification: agent harness, with runtime containment as a distinguishing trait.

SWE-agent has as its central contribution the agent-computer interface: an interface designed so that the agent perceives and alters the software environment effectively, with an action loop on the repository [55]. It satisfies T1 (loop), T2 (the agent-computer interface is the materialization of T2), T3 (management of what the agent sees of the repository), and T4 (limits and an action format that structure the behavior). Classification: agent harness, and it is the case that best illustrates T2 in isolation. We should keep SWE-agent apart from SWE-bench, the eval harness against which systems such as it are measured [19]: the first executes, the second evaluates.

Classifying only systems already known to be harnesses confirms, it does not discriminate. The six above were chosen because they are harnesses, so passing the test was expected. The discriminating power appears when the test *excludes*, and the most instructive case is a real, named system that is commonly grouped among AI coding tools. The first edge case is classic inline autocomplete, that is, code suggestion from the cursor, such as the completion feature of GitHub Copilot or Tabnine, scoped strictly to the inline completion feature.[11] In this feature, the system suggests the continuation of the code without keeping task state. It reads editor signals but does not alter the repository, so it does not satisfy T2, and it does not close a reasoning and action loop over the repository (fails T1) and neither verifies the result nor contains the action (fails T4). The test excludes it, and rightly so: no one operates inline completion by handing it a multi-step task and expecting it to finish on its own. The second is a fixed orchestration pipeline, which always runs retrieve, then generate, then format, without letting the observation of one step alter the next. It has tools (T2), but its context selection is fixed rather than driven by the current observation (fails T3), and the course is a fixed graph, not an adaptive loop (fails T1). The test excludes it too. The two cases show that T1 through T4 is not a stamp that approves everything. It rejects plausible systems that lack the adaptive loop or the control, which is what separates a harness from an assistant or a pipeline.

Figure 4 maps the six systems against the anatomy components of Section 4 and separates the necessary core from the optional qualifiers. Three things stand out. The four core elements (loop, tools, context, control) appear in all six, and that should not be over-interpreted: since

[9] Aider documentation: https://aider.chat.

[10] Cline documentation and repository: https://github.com/cline/cline.

[11] Official product pages: GitHub Copilot https://github.com/features/copilot and Tabnine https://www.tabnine.com. The analysis here is restricted to the inline completion feature (suggestion from the cursor); it does not cover the agentic modes (*agent mode*) these products have since come to offer, which could indeed satisfy the test and constitute a harness.

the six were chosen for already being harnesses, the presence of the core is almost tautological, and the real discriminating power came from the edge cases the test excludes, not from the six passing. The optional components, by contrast, vary, and that is where the systems differentiate themselves in design: some emphasize sandbox containment (OpenHands), others auditability through version control (Aider), others runtime guardrails (Claude Code, Codex CLI). And the form of control (T4) is the axis of greatest variation, which anticipates the research agenda of Section 7. Table 4 closes the classification.

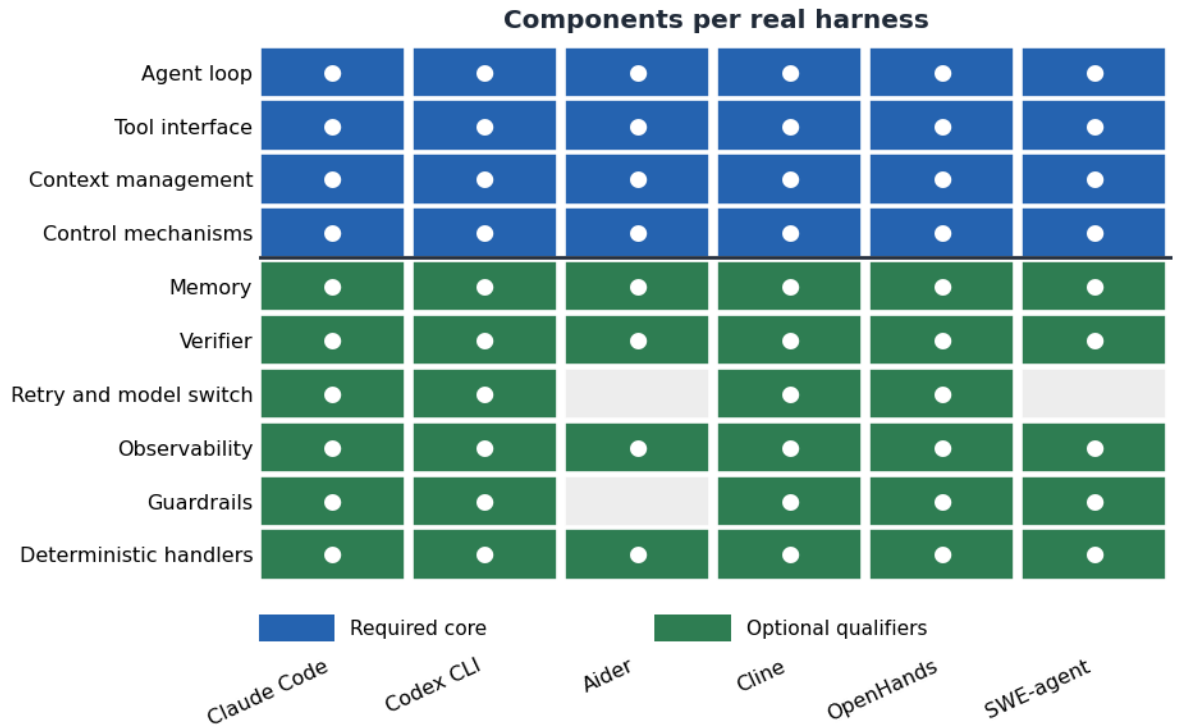

Figure 4: Component map per real harness. The rows are the anatomy components (necessary core above, optional qualifiers below); the columns are the six harnesses. The four core elements are present in all systems, as expected of a membership condition; the optional qualifiers vary and reveal design differences, above all in the form of control.

System	T1	T2	T3	T4	Distinguishing control trait (T4)
Claude Code	yes	yes	yes	yes	Runtime guardrails; confirmation of destructive actions
Codex CLI	yes	yes	yes	yes	Graded permission modes
Aider	yes	yes	yes	yes	Auditability and reversal via version control
Cline	yes	yes	yes	yes	Human approval before sensitive actions
OpenHands	yes	yes	yes	yes	Sandboxed execution (deterministic containment)
SWE-agent	yes	yes	yes	yes	Structured action interface with limits

Table 4: Classification of the six real harnesses by the T1 to T4 test. All belong to the concept; the final column shows that the form of control (T4) is where the designs diverge most.

7 Design Axes and Research Agenda (RQ5)

The six harnesses agree on the core and diverge on the qualifiers, as Section 6 showed. These divergences are not noise. They are design choices about real tensions, and each tension opens a research front. We answer RQ5 by organizing the field along four tension axes and deriving, from each, open questions. Figure 5 places the six systems on these axes qualitatively, based on the prior classification.

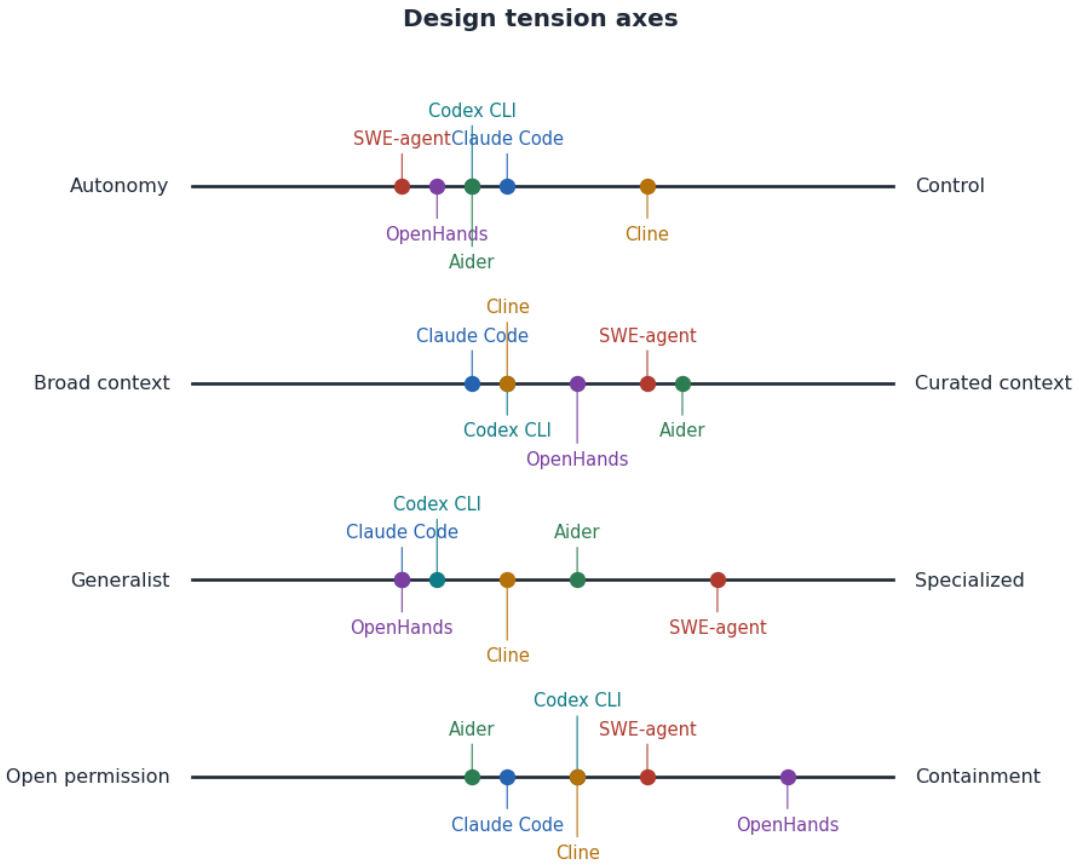

Figure 5: The six real harnesses placed qualitatively on four design tension axes: autonomy versus control, broad versus curated context, generalist versus specialized, and open permission versus containment. The positions derive from the classification in Section 6 and illustrate that the concept is single while the design space is wide.

Axis 1: autonomy versus control

The more the harness lets the agent decide on its own, the more useful and the more risky it becomes. Cline asks for human approval at every sensitive action and trades speed for safety; Codex CLI grades its permission modes and attempts a configurable middle ground. Agent failures, from prompt injection to adversarial behavior, show that autonomy without control independent of the model is brittle [27, 10, 49]. Two questions stay open. How do we measure the optimal point between autonomy and supervision for a task class? And how do we design verifiers that scale with autonomy, rather than bottlenecking it [28]?

Axis 2: broad versus curated context

A harness can dump the whole repository into the context or actively curate what the model sees. Models degrade when useful information is diluted in long context, which favors curation [25]; context engineering became a discipline precisely because of that [30]. But curating costs. It requires a repository map (Aider), well-calibrated retrieval (RAG) [20], and a memory that decides what to retain [32, 48]. Two questions stay open. Which context-curation policies maximize performance per token? And how do we evaluate context management in isolation from the underlying model?

Axis 3: generalist versus specialized

Generalist harnesses (OpenHands, Claude Code) seek to cover many tasks; specialized ones couple verifiers and handlers to the domain. The evidence suggests that effective control is problem-specific: the verifier that checks one business rule does not serve another. Two questions stay open. How much of the harness is reusable across domains and how much must be bespoke? And is there a generalist control core that composes with domain extensions?

Axis 4: open permission versus containment

At the open extreme, the agent runs with the user's privileges; at the contained extreme, it operates in a sandbox (OpenHands) with a restricted surface. Deterministic containment is the strongest control because it does not depend on the model's cooperation, unlike the prompt guardrail, which is the weakest [3, 18]. Two questions stay open. How do we compose strong containment with high utility, without the sandbox preventing the legitimate task? And which containment patterns transfer across harnesses?

Three findings cut across the four axes in a cross-cutting way. Control (T4) is what most distinguishes designs and what the formal literature has least consolidated; it is a particularly open front. The separation between model and harness carries a strategic consequence: the better the harness, the less the application depends on a single large and expensive model, since model switching becomes a control mechanism, not a rewrite [60]. If this separation between model and harness holds, it is plausible that the engineering differential may shift from the model toward the harness, since the harness is problem-specific; but this is a conjecture, and testing it empirically is left as future work. And the evaluation of harnesses, today, measures above all the model-harness pair through task benchmarks [19, 26, 57]; an evaluation that isolates the harness's contribution, controlling for the model, is missing. That is, perhaps, a central methodological gap that this article's definition helps to make formulable: one can only measure a harness's contribution after knowing, precisely, what it is.

8 Conclusion

The term *agent harness* became central to software engineering with generative AI before it became a definition. We reversed that order. We reconstructed the genealogy of the term, from the horse's tack to the test harness, to the evaluation harness, and to the agent harness. The thread is single: a harness is the infrastructure that channels a force to produce useful work under control. What makes the agentic sense distinct is one thing: this control acts at runtime. On that base, we define the harness by four conditions, agent loop, tool interface, context management, and control mechanisms, and we turn the definition into an inclusion and exclusion test that decides, given a concrete system, whether or not it is an agent harness.

The test is an instrument, not a label. At the boundary, it separates the harness from an agent framework, an SDK, an IDE plugin, an eval harness, and an orchestrator; each neighbor fails one condition you can point to. In practice, we classified six real harnesses, Claude Code, Codex CLI, Aider, Cline, OpenHands, and SWE-agent. All share the core. They diverge on the qualifiers, and they diverge above all in the form of control. From those divergences we drew a research agenda organized along four design tension axes.

At bottom, the contribution is one of conceptual hygiene: giving an overloaded term a meaning that includes and excludes cases. We wager that cumulative science about trustworthy agents depends, first of all, on a vocabulary that does not call too many things a *harness*. The definition opens the question that motivated it and that it does not yet answer: how do we measure a harness's contribution by isolating it from the model it wraps? That is the natural continuation

of this work. Knowing what the harness is was the necessary step toward knowing, next, how much it is worth.

Declaration on the Use of Generative AI

The author conducted the research and wrote the manuscript. During the preparation of this study, however, the author used Grammarly tools to improve textual agreement and Claude Opus 4.8 to support text structuring and translation into English. After using these tools/services, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

[1] Daman Arora, Atharv Sonwane, Nalin Wadhwa, Abhav Mehrotra, Saiteja Utpala, Ramakrishna Bairi, Aditya Kanade, and Nagarajan Natarajan. MASAI: Modular Architecture for Software-engineering AI Agents, 2024. Preprint, arXiv. doi:10.48550/arXiv.2406.11638.

[2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional AI: Harmlessness from AI Feedback, 2022. Preprint, arXiv. doi:10.48550/arXiv.2212.08073.

[3] Clark Barrett, Brad Boyd, Elie Burzstein, Nicholas Carlini, Brad Chen, Jihye Choi, Amrita Roy Chowdhury, Mihai Christodorescu, Anupam Datta, Soheil Feizi, Kathleen Fisher, Tatsunori Hashimoto, Dan Hendrycks, Somesh Jha, Daniel Kang, Florian Kerschbaum, Eric Mitchell, John Mitchell, Zulfikar Ramzan, Khawaja Shams, Dawn Song, Ankur Taly, and Diyi Yang. Identifying and Mitigating the Security Risks of Generative AI, 2023. Preprint, arXiv. doi:10.48550/arXiv.2308.14840.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners, 2020. Preprint, arXiv. doi:10.48550/arXiv.2005.14165.

[5] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. Sparks of Artificial General Intelligence: Early experiments with GPT-4, 2023. Preprint, arXiv. doi:10.48550/arXiv.2303.12712.

[6] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie. A Survey on Evaluation of Large Language Models, 2023. Preprint, arXiv. doi:10.48550/arXiv.2307.03109.

[7] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. CodeR: Issue Resolving with Multi-Agent and Task Graphs, 2024. Preprint, arXiv. doi:10.48550/arXiv.2406.01304.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating Large Language Models Trained on Code, 2021. Preprint, arXiv. doi:10.48550/arXiv.2107.03374.

[9] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors, 2023. Preprint, arXiv. doi:10.48550/arXiv.2308.10848.

[10] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents, 2024. Preprint, arXiv. doi:10.48550/arXiv.2406.13352.

[11] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a Generalist Agent for the Web, 2023. Preprint, arXiv. doi:10.48550/arXiv.2306.06070.

[12] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. Self-collaboration Code Generation via ChatGPT, 2023. Preprint, arXiv. doi:10.48550/arXiv.2304.07590.

[13] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang, Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks, 2024. Preprint, arXiv. doi:10.48550/arXiv.2411.04468.

[14] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. AgentScope: A Flexible yet Robust Multi-Agent Platform, 2024. Preprint, arXiv. doi:10.48550/arXiv.2402.14034.

[15] Thomas R. Gruber. Toward principles for the design of ontologies used for knowledge sharing. *International Journal of Human-Computer Studies*, 43(5-6):907–928, 1995. doi:10.1006/ijhc.1995.1081.

[16] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework, 2023. Preprint, arXiv. doi:10.48550/arXiv.2308.00352.

[17] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large Language Models for Software Engineering: A Systematic Literature Review, 2023. Preprint, arXiv. doi:10.48550/arXiv.2308.10620.

[18] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations, 2023. Preprint, arXiv. doi:10.48550/arXiv.2312.06674.

[19] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, 2023. Preprint, arXiv. doi:10.48550/arXiv.2310.06770.

[20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, 2020. Preprint, arXiv. doi:10.48550/arXiv.2005.11401.

[21] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society, 2023. Preprint, arXiv. doi:10.48550/arXiv.2303.17760.

[22] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs, 2023. Preprint, arXiv. doi:10.48550/arXiv.2304.08244.

[23] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation, 2023. Preprint, arXiv. doi:10.48550/arXiv.2305.01210.

[24] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. Large Language Model-Based Agents for Software Engineering: A Survey, 2024. Preprint, arXiv. doi:10.48550/arXiv.2409.02977.

[25] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts, 2023. Preprint, arXiv. doi:10.48550/arXiv.2307.03172.

[26] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as Agents, 2023. Preprint, arXiv. doi:10.48550/arXiv.2308.03688.

[27] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and Yang Liu. Prompt Injection attack against LLM-integrated Applications, 2023. Preprint, arXiv. doi:10.48550/arXiv.2306.05499.

[28] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback, 2023. Preprint, arXiv. doi:10.48550/arXiv.2303.17651.

[29] Tula Masterman, Sandi Besen, Mason Sawtell, and Alex Chao. The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling: A Survey, 2024. Preprint, arXiv. doi:10.48550/arXiv.2404.11584.

[30] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A Survey of Context Engineering for Large Language Models, 2025. Preprint, arXiv. doi:10.48550/arXiv.2507.13334.

[31] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. Preprint, arXiv. doi:10.48550/arXiv.2203.02155.

[32] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems, 2023. Preprint, arXiv. doi:10.48550/arXiv.2310.08560.

[33] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior, 2023. Preprint, arXiv. doi:10.48550/arXiv.2304.03442.

[34] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs, 2023. Preprint, arXiv. doi:10.48550/arXiv.2305.15334.

[35] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative Agents for Software Development, 2023. Preprint, arXiv. doi:10.48550/arXiv.2307.07924.

[36] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, Yi Ren Fung, Yusheng Su, Huadong Wang, Cheng Qian, Runchu Tian, Kunlun Zhu, Shihao Liang, Xingyu Shen, Bokai Xu, Zhen Zhang, Yining Ye, Bowen Li, Ziwei Tang, Jing Yi, Yuzhang Zhu, Zhenning Dai, Lan Yan, Xin Cong, Yaxi Lu, Weilin Zhao, Yuxiang Huang, Junxi Yan, Xu Han, Xian Sun, Dahai Li, Jason Phang, Cheng Yang, Tongshuang Wu, Heng Ji, Zhiyuan Liu, and Maosong Sun. Tool Learning with Foundation Models, 2023. Preprint, arXiv. doi:10.48550/arXiv.2304.08354.

[37] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs, 2023. Preprint, arXiv. doi:10.48550/arXiv.2307.16789.

[38] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails, 2023. Preprint, arXiv. doi:10.48550/arXiv.2310.10501.

[39] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools, 2023. Preprint, arXiv. doi:10.48550/arXiv.2302.04761.

[40] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face, 2023. Preprint, arXiv. doi:10.48550/arXiv.2303.17580.

[41] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, 2023. Preprint, arXiv. doi:10.48550/arXiv.2303.11366.

[42] Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, Ye Tian, and Sujian Li. RestGPT: Connecting Large Language Models with Real-World RESTful APIs, 2023. Preprint, arXiv. doi:10.48550/arXiv.2306.06624.

[43] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive Architectures for Language Agents, 2023. Preprint, arXiv. doi:10.48550/arXiv.2309.02427.

[44] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A Survey on Large Language Model based Autonomous Agents, 2023. Preprint, arXiv. doi:10.48550/arXiv.2308.11432.

[45] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable Code Actions Elicit Better LLM Agents, 2024. Preprint, arXiv. doi:10.48550/arXiv.2402.01030.

[46] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents, 2024. Preprint, arXiv. doi:10.48550/arXiv.2407.16741.

[47] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models, 2022. Preprint, arXiv. doi:10.48550/arXiv.2203.11171.

[48] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory, 2024. Preprint, arXiv. doi:10.48550/arXiv.2409.07429.

[49] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How Does LLM Safety Training Fail?, 2023. Preprint, arXiv. doi:10.48550/arXiv.2307.02483.

[50] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent Abilities of Large Language Models, 2022. Preprint, arXiv. doi:10.48550/arXiv.2206.07682.

[51] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, 2022. Preprint, arXiv. doi:10.48550/arXiv.2201.11903.

[52] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation, 2023. Preprint, arXiv. doi:10.48550/arXiv.2308.08155.

[53] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. The Rise and Potential of Large Language Model Based Agents: A Survey, 2023. Preprint, arXiv. doi:10.48550/arXiv.2309.07864.

[54] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents, 2024. Preprint, arXiv. doi:10.48550/arXiv.2407.01489.

[55] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, 2024. Preprint, arXiv. doi:10.48550/arXiv.2405.15793.

[56] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains?, 2024. Preprint, arXiv. doi:10.48550/arXiv.2410.03859.

[57] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, 2024. Preprint, arXiv. doi:10.48550/arXiv.2406.12045.

[58] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models, 2023. Preprint, arXiv. doi:10.48550/arXiv.2305.10601.

[59] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models, 2022. Preprint, arXiv. doi:10.48550/arXiv.2210.03629.

[60] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. Survey on Evaluation of LLM-based Agents, 2025. Preprint, arXiv. doi:10.48550/arXiv.2503.16416.

[61] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous Program Improvement, 2024. Preprint, arXiv. doi:10.48550/arXiv.2404.05427.

[62] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A Survey on the Memory Mechanism of Large Language Model based Agents, 2024. Preprint, arXiv. doi:10.48550/arXiv.2404.13501.

[63] Ziyin Zhang, Chaoyu Chen, Bingchang Liu, Cong Liao, Zi Gong, Hang Yu, Jianguo Li, and Rui Wang. Unifying the Perspectives of NLP and Software Engineering: A Survey on Language Models for Code, 2023. Preprint, arXiv. doi:10.48550/arXiv.2311.07989.

[64] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A Survey of Large Language Models, 2023. Preprint, arXiv. doi:10.48550/arXiv.2303.18223.

[65] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A Realistic Web Environment for Building Autonomous Agents, 2023. Preprint, arXiv. doi:10.48550/arXiv.2307.13854.

[66] Wangchunshu Zhou, Yuchen Eleanor Jiang, Long Li, Jialong Wu, Tiannan Wang, Shi Qiu, Jintian Zhang, Jing Chen, Ruipu Wu, Shuai Wang, Shiding Zhu, Jiyu Chen, Wentao Zhang, Xiangru Tang, Ningyu Zhang, Huajun Chen, Peng Cui, and Mrinmaya Sachan. Agents: An Open-source Framework for Autonomous Language Agents, 2023. Preprint, arXiv. doi:10.48550/arXiv.2309.07870.

[67] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models, 2023. Preprint, arXiv. doi:10.48550/arXiv.2307.15043.